



Z&L ScaleTech

TP2 – Flutter - Application mobile de pointage

Formation : Introduction au développement mobile

Sicilia Toury
10/02/2026

TP1 : Flutter – Application mobile de pointage

Temps estimé 5h

Contexte :

Une entreprise souhaite disposer d'une application mobile permettant aux employés de se pointer chaque matin lorsqu'ils arrivent au travail.

L'objectif de ce TP est de développer une application mobile avec Flutter et Android Studio permettant :

- Aux employés de se connecter
- D'enregistrer leur heure d'arrivée
- De consulter l'historique de leurs pointages

Ce projet vous permettra de mettre en pratique la création d'interfaces Flutter, la navigation entre écrans et la gestion de données.

Objectif :

À la fin de ce TP vous serez capable de :

- Créer une application Flutter
- Structurer un projet Flutter
- Créer plusieurs écrans
- Mettre en place la navigation entre écrans
- Manipuler la date et l'heure
- Stocker et afficher des données

Prérequis :

- Avoir installé **Android Studio**
- Avoir installé **Flutter SDK**
- Savoir lancer une application Flutter
- Connaître les bases du langage **Dart**

Travail à réaliser :

- Fonctionnalités attendues :
- Un écran de connexion
- Un écran d'accueil
- Un bouton permettant de se pointer
- Un système enregistrant la date et l'heure du pointage
- Un écran affichant l'historique des pointages

Etape 1 : Création du projet

1. Ouvrir Android Studio
2. Créer un nouveau projet Flutter
3. Nommer le projet : pointage_app
4. Lancer l'application pour vérifier que tout fonctionne

Etape 2 : Organisation du projet

Organisez votre dossier **lib** afin de séparer les différentes parties de l'application.

Vous devrez créer plusieurs dossiers afin de structurer correctement votre projet.

Exemples de catégories à créer :

- Écrans de l'application
- Modèles de données
- Services ou logique métier

Votre projet devra contenir au minimum :

- Un fichier principal
- Un écran de connexion
- Un écran d'accueil
- Un écran d'historique
- Un modèle représentant un pointage

Etape 3 : Ecran de connexion

Créer un premier écran permettant à un employé de saisir son nom.

L'interface doit contenir :

- Un titre ou logo de l'application
- Un champ de saisie pour le nom de l'employé
- Un bouton pour se connecter

Lorsque l'utilisateur clique sur le bouton :

- Il est redirigé vers l'écran d'accueil
- Son nom est transmis à l'écran suivant

Etape 4 : Ecran d'accueil

L'écran d'accueil doit afficher :

- Un message de bienvenue avec le nom de l'utilisateur
- Un bouton pour effectuer un pointage
- Un bouton pour consulter l'historique

Lorsque l'utilisateur appuie sur **Se pointer** :

- L'application enregistre la date et l'heure actuelles
- Un message de confirmation s'affiche

Etape 5 : Modèle de données

Créer un modèle représentant un pointage.

Un pointage doit contenir au minimum :

- Le nom de l'employé
- La date
- L'heure

Ce modèle servira à stocker les informations de présence.

Etape 6 : Enregistrement des pointages

Mettre en place un système permettant de :

- Enregistrer chaque pointage
- Conserver les données
- Récupérer les données plus tard

Vous pouvez utiliser un système de stockage local pour sauvegarder les informations.

Etape 7 : Historique des pointages

Créer un écran permettant d'afficher l'historique des pointages.

Cet écran doit afficher :

- Une liste des pointages
- La date
- L'heure

Chaque pointage doit apparaître sous forme d'élément dans une liste.

Etape 8 Bonus : Amélioration de l'expérience utilisateur

Améliorer l'application en ajoutant :

- Des icônes
- Une mise en page claire
- Des espacements corrects
- Une interface agréable

Résultat attendu :

Votre application doit permettre :

- La connexion d'un employé
- L'enregistrement d'un pointage
- L'affichage de l'historique

L'application doit être fonctionnelle sur un émulateur Android.

Je testerai chaque appli pour voir son fonctionnement.

Consigne technique

STRUCTUREZ VOTRE PROJET DES LE DEBUT :

- 1 Ecran = 1 fichier

Conseil : Si ton fichier dépasse 200 lignes, c'est qu'il y a un problème

PENSEZ NAVIGATION DES LE DEPART, AVANT DE CODER, REFLECHISSEZ :

- Quels écrans ?
- Dans quel ordre ?
- Quelles données circulent ?
- Exemple : Login → Home → Historique

Conseil : Un bon dev anticipe les flux avant de coder.

UTILISEZ LES BONS WIDGETS

Ne mettez pas tout dans des Column ou Container.

À privilégier :

- ListView → pour les listes
- Scaffold → structure de page
- Expanded → gestion de l'espace

Conseil : Choisir le bon widget simplifie 50% du code.

Gérez l'état correctement

Quand utiliser quoi :

- StatelessWidget → UI simple
- StatefulWidget → données qui changent
- Exemple : bouton de pointage → nécessite un StatefulWidget

Conseil : Si ton écran change, il doit être Stateful.

MANIPULEZ BIEN LA DATE ET L'HEURE

Attention aux erreurs classiques :

- Format d'affichage
- Stockage brut vs affichage
- Astuce : stocker un DateTime, afficher une version formatée.

Conseil : On stocke propre, on affiche joli.

SAUVEGARDEZ LES DONNEES PROPREMENT

Même en local :

- Évitez les variables globales
- Centralisez la logique
- Bonne pratique : créer un service (ex : PointageService)

Conseil : La logique métier ne doit pas être dans l'UI.

GEREZ LES ERREURS UTILISATEUR

Ne laissez pas l'utilisateur faire n'importe quoi :

- Champ vide
- Double pointage
- À faire : validation avant action, messages clairs

Conseil : Une app robuste anticipe les erreurs.

TESTEZ SOUVENT

Ne codez pas tout d'un coup.

Méthode :

- Un écran
- Test
- Amélioration
- Suivant

Conseil : Un bug détecté tôt = 10x plus rapide à corriger.

SOIGNEZ UN MINIMUM L'UI

Même sans être designer :

- Alignements propres
- Espacements cohérents
- Boutons lisibles
- Astuce simple : utilisez SizedBox pour espacer

Conseil : Une app moche donne l'impression qu'elle bug.

LISEZ LES ERREURS FLUTTER

Ne les ignorez pas !

Bonne pratique :

- Lire le message
- Identifier le fichier
- Comprendre la cause

Conseil : Flutter te dit presque toujours où est le problème.

« Je ne veux pas juste une application qui fonctionne.

Je veux une application propre, compréhensible et maintenable »

Partie 3 : Compte rendu obligatoire

Vous devez rendre :

- Le projet Flutter complet
- Une capture d'écran de l'application
- Une capture d'écran du code montrant les fonctionnalités
- Un document expliquant votre application

Un travail sans compte rendu ou sans screenshots = travail incomplet